Chapter 1

L2 - Relational Model & SQL Basics

1.1 Relational model

The relational model is the foundation of modern database systems. It represents all data as tables and provides a clean, mathematically grounded framework for querying and constraining data.

1.1.1 Core concepts

A **database** is a set of named *relations* (*named tables with rows and columns*). Each attribute carries a **type** (domain): integer, real, string, file formats, enumerated, etc.

Tuple. A single row in a relation, containing one value for each attribute. All tuples in a relation are distinct and have no inherent order.

Running example:

Students

sid	name	login	age	gpa
50000	Dave	dave@cs	19	3.3
53666	Jones	jones@cs	18	3.4
53688	Smith	smit@ee	18	3.2
...	...	...	...	...

Colleges

name	location	strength
MIT	USA	10000
Oxford	UK	22000
EPFL	CH	9000
...	...	...

1.1.2 Schema vs. instance

A relation has two complementary aspects: its *schema* (*structural definition with attribute names and types*) and its content at a given moment.

Example schema: `Students(sid: string, name: string, login: string, age: integer, gpa: real)`.

Instance. The actual set of tuples stored in a relation at a given point in time.

Two measures describe an instance:

- **Cardinality**: the number of tuples (rows).
- **Arity** (or degree): the number of attributes (columns).

1.1.3 Null values

Not every attribute value is always known. A student who has not received any grades yet has no meaningful GPA.

Null value. A special marker representing an "unknown" or "undefined" attribute value. Nulls are not the same as zero or an empty string.

1.2 Keys and integrity constraints

Keys (*attributes that uniquely identify tuples*) enforce uniqueness and link relations together. Here we refine the hierarchy of key types and formalize integrity constraints.

1.2.1 Key hierarchy

Superkey. A set of attributes such that no two distinct tuples can share the same values in all those fields. Any superset of a key is also a superkey.

Candidate key. A *minimal* superkey: it is a superkey, and no proper subset of its fields is also a superkey.

Primary key. The candidate key chosen by the database administrator (DBA) as *the* key of the relation.

Example with **Students**(sid, name, login, age, gpa): both `sid` and `login` are candidate keys. The DBA picks `sid` as the primary key.

- Every superset of `sid` (e.g., {`sid, name`}, {`sid, login, gpa`}, ...) is a superkey but not a candidate key.
- `sid` alone and `login` alone are the only candidate keys.

1.2.2 Integrity constraints

A key is an **integrity constraint** (IC): a condition that must hold on every valid database instance.

Foreign keys (*attributes referencing the primary key of another relation*) establish **referential integrity**: every foreign-key value must match an existing key in the referenced relation.

1.3 SQL: Structured Query Language

SQL is the standard language for interacting with relational databases. It splits into two sub-languages.

1.3.1 DDL and DML

Data Definition Language (DDL). The subset of SQL used to create, modify, and delete relations, specify constraints, and administer users and security.

Data Manipulation Language (DML). The subset of SQL used to query, insert, update, and delete tuples.

1.3.2 Most common SQL commands

```
CREATE TABLE <name> (<field> <domain>, ...)
INSERT INTO <name> (<field names>) VALUES (<field values>)
DELETE FROM <name> WHERE <condition>
UPDATE <name> SET <field name> = <value> WHERE <condition>
SELECT <fields> FROM <name> WHERE <condition>
```

1.3.3 Creating relations

A `CREATE TABLE` statement defines a new relation by listing each attribute and its domain. The type of each field is enforced by the DBMS whenever tuples are added or modified.

Students – holds data about students:

```
CREATE TABLE Students
  (sid CHAR(20),
   name CHAR(20),
   login CHAR(10),
   age INTEGER,
   gpa FLOAT)
```

Enrolled – holds course enrollments:

```
CREATE TABLE Enrolled
  (sid CHAR(20),
   cid CHAR(20),
   grade CHAR(2))
```

1.3.4 Adding and deleting tuples

Insert a single tuple:

```
INSERT INTO Students (sid, name, login, age, gpa)
VALUES ('53688', 'Smith', 'smith@cs', 18, 3.2)
```

Delete all tuples satisfying a condition:

```
DELETE FROM Students S WHERE S.name = 'Smith'
```

1.3.5 Keys in SQL

The `PRIMARY KEY` clause declares the primary key; `UNIQUE` marks additional candidate keys.

```
CREATE TABLE Students
  (sid CHAR(20),
   name CHAR(20),
   login CHAR(10),
   age INTEGER,
   gpa FLOAT,
   PRIMARY KEY(sid))
```

```
CREATE TABLE Person
  (ssn CHAR(9),
   name CHAR(20),
   licence# CHAR(10),
   PRIMARY KEY(ssn),
   UNIQUE(licence#))
```

Choosing keys carefully

Keys must be used carefully: they encode real-world constraints. Consider `Enrolled`: "for a given student and course, there is a single grade."

Correct – composite PK on `(sid, cid)`:

```
CREATE TABLE Enrolled
  (sid CHAR(20),
   cid CHAR(20),
   grade CHAR(2),
   PRIMARY KEY(sid, cid))
```

Wrong – PK on `sid` alone with `UNIQUE(cid, grade)` encodes a different constraint: "students can take only one course, and no two students in a course receive the same grade."

1.3.6 Foreign keys and referential integrity

Foreign keys (*fields referencing the primary key of another relation*) act as logical pointers between relations. If all foreign-key constraints are enforced, the database achieves **referential integrity** (no dangling references).

Students

sid	name	login	age	gpa
50000	Dave	dave@cs	19	3.3
53666	Jones	jones@cs	18	3.4
53688	Smith	smit@ee	18	3.2
...	...	...	...	...

Enrolled

cid	sid	grade
Carnatic101	53666	C
Reggae203	50000	B
Topology112	53666	A
...	...	...

Here `Enrolled.sid` is a foreign key referencing `Students.sid`: every enrolled student must actually exist in `Students`.

Enforcing referential integrity

The DBMS must decide what to do when a foreign-key constraint is threatened:

- **Insert** an Enrolled tuple with a non-existent `sid` → reject the insert.
- **Delete** a Students tuple that is referenced by Enrolled → several policies:
  - cascade-delete all Enrolled tuples that refer to it,
  - disallow the deletion entirely,
  - set the referencing `sid` to a default value,
  - set it to `NULL` (denoting "unknown" or "inapplicable").
- **Update** the primary key of a Students tuple → same policies apply.

1.3.7 Integrity constraints

Integrity constraint (IC). A condition that must be true for *any* instance of the database (e.g., domain constraints, key constraints, foreign-key constraints). ICs are specified when the schema is defined and checked whenever relations are modified.

Legal instance. An instance of a relation that satisfies all specified ICs. The DBMS should not allow illegal instances.

When the DBMS enforces ICs, stored data is more faithful to real-world meaning and data-entry errors are caught early.

1.4 Relational query languages

Query language (QL). A language that allows manipulation and retrieval of data from a database. Unlike programming languages, QLs are not Turing-complete: they are designed for easy, efficient access to large data sets, not general computation.

The relational model supports simple yet powerful QLs with a strong formal foundation based on logic, enabling significant optimization.

1.4.1 Formal relational query languages

Two mathematical query languages form the basis for real languages like SQL:

Relational Algebra. An *operational* language: it specifies *how* to compute a result step by step. Useful for representing execution plans inside database engines.

Relational Calculus. A *declarative* language: it specifies *what* the result should look like, not how to compute it. Non-procedural.

Codd's Theorem. Relational algebra and relational calculus are equivalent in expressive power: for every query in one, there is an equivalent query in the other.

Understanding both is key to understanding SQL and query processing.

1.4.2 Preliminaries

A query is applied to *relation instances*, and the result of a query is also a relation instance.

- The *schemas* of input relations are fixed (but the query runs over any legal instance).
- The schema of the *result* is also fixed, determined by the query language constructs.

Two notations are used:

- **Positional notation**: easier for formal definitions.
- **Named-field notation**: more readable; both are used in SQL.

1.4.3 Example schema and instances

We use a university database throughout the relational algebra examples:

```
instructor  ID, name, dept_name, salary
course      course_id, title, dept_name, credits
teaches     ID, course_id, sec_id, semester, year
section     course_id, sec_id, semester, year, building, room_number, time_slot_id
student     ID, name, dept_name, tot_cred
takes       ID, course_id, sec_id, semester, year, grade
```

Running example instance of `instructor`:

instructor

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

1.4.4 Five basic operations

Relational algebra has five primitive operators, split into two groups:

Unary operators (one input relation):

- **Selection** (σ): output a subset of *rows* (horizontal filtering).
- **Projection** (π): output a subset of *columns* (vertical filtering).

Binary operators (two input relations):

- **Cross-product** ($\times$): concatenate every row of the first relation with every row of the second.
- **Set-difference** ($-$): output tuples in R that are not in S.
- **Union** ($\cup$): output tuples in R or in S (or both).

Since each operation returns a relation, **operations can be composed**.

1.4.5 Selection (σ)

Selection outputs only the rows that satisfy a given condition. The output schema is the same as the input.

$$\sigma_{\text{condition}}(relation)$$

Without condition

The simplest expression $\sigma(instructor)$ returns all rows unchanged (equivalent to `SELECT * FROM instructor`).

With a single condition

$$\sigma_{dept\_name=\text{“Physics”}}(instructor)$$

```
SELECT * FROM instructor WHERE dept_name = 'Physics'
```

ID	name	dept_name	salary
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

With a compound condition

Conditions can be combined with $\wedge$ (and), $\vee$ (or), and $\neg$ (not):

$$\sigma_{dept\_name=\text{“Physics”} \wedge salary>90000}(instructor)$$

```
SELECT * FROM instructor WHERE dept_name = 'Physics' AND salary > 90000
```

ID	name	dept_name	salary
22222	Einstein	Physics	95000

1.4.6 Projection (π)

Projection retains only the attributes in the projection list. The output schema contains exactly those attributes.

$$\pi_{\text{attribute list}}(relation)$$

$$\pi_{ID,\ name,\ salary}(instructor)$$

corresponds to:

```
SELECT ID, name, salary FROM instructor
```

The result has three columns instead of four (`dept_name` is dropped), but all 12 rows are kept because each row is still distinct.

Duplicate elimination

In relational algebra, projection **eliminates duplicates** because the result must be a set of tuples.

$$\pi_{dept\_name}(instructor)$$

reduces the 12 rows to 7 distinct department names. In SQL, duplicates are kept by default; to match the algebra, use `SELECT DISTINCT`:

```
SELECT DISTINCT dept_name FROM instructor
```

1.4.7 Composing operators

Since every operator returns a relation, the output of one can feed into another. Selection followed by projection gives specific columns from filtered rows:

$$\pi_{name}(\sigma_{dept\_name=\text{“Physics”}}(instructor))$$

```
SELECT name FROM instructor WHERE dept_name = 'Physics'
```

name
Einstein
Gold

First σ filters to Physics instructors, then π keeps only the `name` column. The order matters: projecting first would discard the column needed for selection.

1.4.8 Rename operator (ρ)

The rename operator (ρ) renames a relation or its attributes. Since algebraic expressions can yield unnamed relations natively, renaming is essential to avoid naming conflicts or to self-reference.

- Renames the list of attributes in the form oldname $\rightarrow$ newname or position $\rightarrow$ newname.
- Can be used to rename the output relation itself entirely.
- **Output schema**: same as input except for the renamed attributes.
- **Output instances**: returns the exact same tuples as the input.

Renaming the relation itself to i:

$$\rho_i(instructor)$$

Renaming the ID attribute to $instructor.ID$:

$$\rho_{ID \rightarrow instructor.ID}(instructor)$$

In SQL, this is achieved using the `AS` keyword in the `SELECT` or `FROM` clauses:

```
SELECT i.name FROM instructor AS i WHERE i.ID = ...
```

1.4.9 Union operator ($\cup$)

The union operator takes two input relations, which **must** be *union-compatible*:

- They must have the same number of fields.
- The "corresponding" fields must have exactly the same type.

Like projection, union in strict relational algebra implies *duplicate elimination*.

$$c1 \cup c2$$

In SQL, `UNION` implicitly eliminates duplicates, while `UNION ALL` is faster but keeps them.

```
(SELECT * FROM c1)
UNION
(SELECT * FROM c2)
```

1.4.10 Set-difference operator ($-$)

The set-difference operator outputs the set of tuples in $c1$ which are not in $c2$.

- The two input relations must also be *union-compatible*.
- Set difference is **not** commutative ($c1 - c2 \neq c2 - c1$).

$$c1 - c2$$

In SQL, this corresponds to the `EXCEPT` clause:

```
(SELECT * FROM c1)
EXCEPT
(SELECT * FROM c2)
```

1.4.11 Compound operators

Alongside the five basic operators ($\sigma, \pi, \times, -, \cup$), there are several additional **compound operators**. These add **no computational power** to the language (they can all be expressed using the basic operators), but they serve as useful, compact shorthands.

Intersection ($\cap$)

Intersection outputs the overlapping tuples from two *union-compatible* relations.

$$R \cap S = R - (R - S)$$

```
(SELECT * FROM c1)
INTERSECT
(SELECT * FROM c2)
```

1.4.12 Cross-product operator ($\times$)

The Cartesian product $S \times R$ pairs every single row of S with every single row of R.

- **Result schema**: has one field per field of S and R, inheriting the field names linearly.
- **Naming conflicts**: Both S and R may have fields sharing the exact same name. In this case, we use the renaming operator (ρ) to disambiguate.

$$instructor \times teaches$$

1.4.13 Join operator ($\bowtie$)

A simple $instructor \times teaches$ associates *every* instructor tuple with *every* teaches tuple. Most of these rows hold information about an instructor paired with a course they never taught!

To get only tuples of instructors and the specific courses they actually taught, we apply a selection:

$$\sigma_{instructor.id=teaches.id}(instructor \times teaches)$$

Because this pattern (σ after $\times$) is overwhelmingly common in databases, it gets its own dedicated operator called a **Join**.

$$instructor \bowtie_{instructor.id=teaches.id} teaches$$

If the matched columns have the identical name in both tables, the condition can be completely omitted. This is called a **Natural Join**.

In SQL, a join is explicitly specified using `JOIN`:

```
SELECT * FROM instructor
JOIN teaches ON instructor.ID = teaches.ID
```

1.4.14 Complex queries

Operators can be chained to answer sophisticated questions. Consider: "Find the names of all instructors in the Music department together with the course title of all the courses that they teach."

This query involves three relations (`instructor`, `teaches`, `course`) and requires joining them before projecting the desired columns.

Two equivalent formulations:

(a) Apply the selection *after* the full join:

$$\pi_{name,\ title}(\sigma_{dept\_name=\text{“Music”}}(instructor \bowtie (teaches \bowtie \pi_{course\_id,\ title}(course))))$$

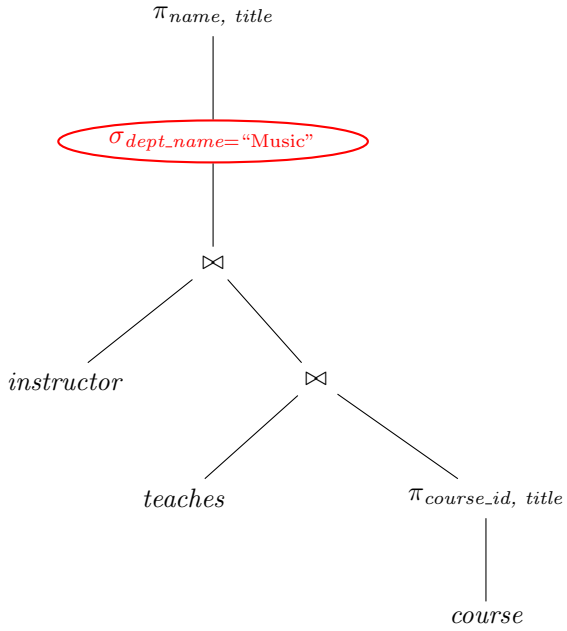
(b) Push the selection *before* the join:

$$\pi_{name,\ title}((\sigma_{dept\_name=\text{“Music”}}(instructor)) \bowtie (teaches \bowtie \pi_{course\_id,\ title}(course)))$$

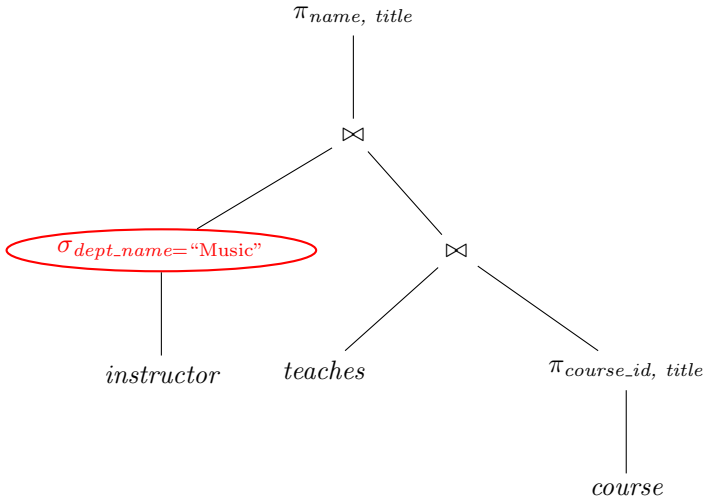
Both return the same result, but formulation (b) is significantly more efficient: the selection filters the instructor relation *before* the expensive join, drastically reducing the number of intermediate tuples.

Query optimization

This illustrates a key principle: **query optimization determines performance**. The DBMS query optimizer automatically rewrites expressions (e.g. pushing selections down) to find an efficient execution plan. The two expression trees below show the difference visually:



(a) Initial: σ applied *after* joining all three relations — processes every tuple.



(b) Optimized: σ pushed *down* to filter *instructor* before the join — far fewer tuples flow upward.

Both trees compute the same result, but (b) is far more efficient because the expensive join operates on a tiny filtered relation instead of the full table.

1.4.15 Division operator (/)

Division is a compound operator useful for expressing "for all" queries (e.g., "find all suppliers who supply *every* part").

Division (A/B). Given relations $A(x, y)$ and $B(y)$, the division A/B returns all values of x such that for **every** tuple b in B, the tuple (x, b) exists in A.

Worked example

Consider relation $A(\texttt{sno}, \texttt{pno})$ and three divisors $B_1 \subseteq B_2 \subseteq B_3$:

$B_1 = \{\text{p2}\}$ $B_2 = \{\text{p2, p4}\}$ $B_3 = \{\text{p1, p2, p4}\}$

$A/B_1 = \{\text{s1, s2, s3, s4}\}$ $A/B_2 = \{\text{s1, s4}\}$ $A/B_3 = \{\text{s1}\}$

A

sno	pno
s1	p1
s1	p2
s1	p3
s1	p4
s2	p1
s2	p2
s3	p2
s4	p2
s4	p4

As B grows (more conditions to satisfy), the result of A/B shrinks: only suppliers matching *every* part in B survive.

Expressing division with basic operators

Division can be expressed using only π, $\times$, and $-$.

Idea: find all x values that are *not* "disqualified". An x is disqualified if pairing it with some $y \in B$ yields a tuple *not* in A.

Disqualified x values:

$$\pi_x((\pi_x(A) \times B) - A)$$

Final result:

$$A/B \;=\; \pi_x(A) \;-\; \pi_x((\pi_x(A) \times B) - A)$$

Step by step with $B_3 = \{\textbf{p1, p2, p4}\}$

- $\pi_{sno}(A) = \{\text{s1, s2, s3, s4}\}$
- $\pi_{sno}(A) \times B_3$ produces all $4 \times 3 = 12$ possible (sno, pno) pairs.
- $(\pi_{sno}(A) \times B_3) - A$ keeps only the *missing* pairs: (s2, p4), (s3, p1), (s3, p4), (s4, p1).
- Projecting on `sno` gives the disqualified suppliers: {s2, s3, s4}.
- $\pi_{sno}(A) - \{\text{s2, s3, s4}\} = \{\text{s1}\}$.